

Ex 2.

```
Q 1 public interface Node extends Remote {  
    void token ( list < Node > list )    t R. ;  
    void request ( Node n )    t R. ;  
}
```

```
Q 2 public class NTNodeImpl implements Node {  
    public static enum state { T, R, CS }  
    private State state = T ;  
    private list < Node > next = new linkedlist ;  
    private Node last = null ;
```

```
    public NodeImpl ( Node node ) {  
        last = node ;  
    }
```

```
public NodeImpl < > { }
```

```

        syn...
    public void lock() {
        state = R;
        if (last != null) {
            new Thread() → last.request(this).start();
            last = null;
            wait;
        }
        state = CS;
    }

```

```

        syn...
    public void request(Node node) {
        if (last == null) {
            if (state != T) {
                next.add('node');
                ...
            }
        }
    }

```

} else {

new Thread (() -> node.token (next)).start;

last = next;

}

} else {

new Thread (() -> last.request (node)).start

last = node;

}

public ^{syn...} void unlock () {

state = T;

if (! next.isEmpty) {

last = next.get ()

NodeImp =

看不清，
根本
看！不！
清！

```

public void unlock() {
    state = T;
    if (!next.isFinished()) {
        last = next.get(next.right);
        Node temp = next.remove(0);
        new Thread(() -> temp.start(next).start());
        next.close();
    }
}

```

11X 4 | 暮雨轻歌

```

public syn. void token (List<Node> list) {
    state = CS;
    next.addAll (list);
    notify ();
}

```

Q3.

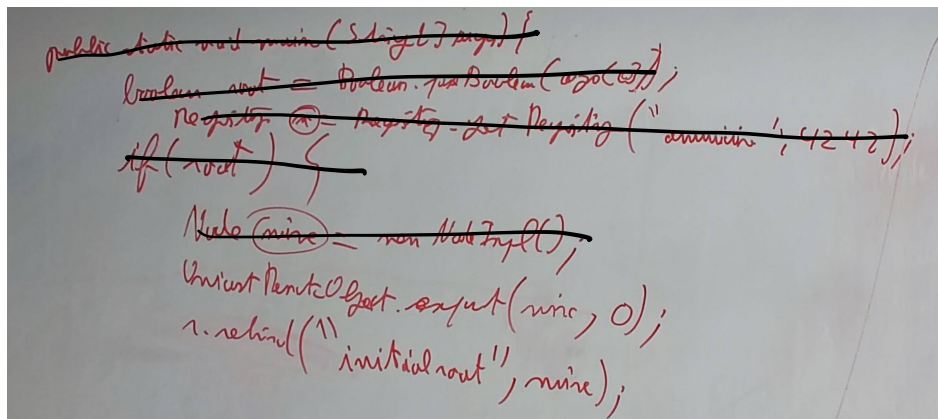
```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        boolean b = Boolean . get Boolean ( args [0] );
```

```
        registry r = Registry . get Registry ( "annuair"  
4242; );  
        if ( b ) {
```

```
            Node node = new NodeImpl;
```



```
public static void main (String[] args) {  
    boolean root = Boolean . get Boolean ( args [0] );  
    registry r = Registry . get Registry ( "annuair"  
4242; );  
    if ( root ) {  
        Node (node) = new NodeImpl();  
        Unicast Remote Object . export ( node , 0 );  
        r . rebind ( "initial root" , node );  
    }  
}
```

```
    } else {
```

```
        node last = null;
```

```
        while ( last == null ) {
```

```
            try {
```

```

        last = (Node).lookup("initialroot");
    } catch (NotBad E ... e) {
        last = null;
        Thread.sleep(1000);
    }
    Node mine? = new NodeImpl(last);
    UnicastRemote.algot.request(mine, 0);
}

```

```

Node root = null;
while (root == null) {
    try {
        root = (Node).lookup("initialroot");
    } catch (NotBadException e) {
        root = null;
        Thread.sleep(1000);
    }
}
Node mine = new NodeImpl(root);
UnicastRemote.algo.request(mine, 0);

```

Ex 3.

Q1. public class Certify {
 im... sera ...
 ✓ }

```

private final String id;
private final String algo;
private final byte[] ;
private final PublicKey key;

```

```

public boolean verify (PublicKey key)
    throw WSE. }

```

```

Signature s = Signature.getInstance (algo);
s.initVerify (key);
s.update (

```

Handwritten code snippet:

```

public class Cardif implements Serializable {
    private final String id;
    private final String algo;
    private final byte[] signature;
    private final PublicKey key;

    public boolean verify (PublicKey key) throws WSE {
        Signature s = Signature.getInstance (algo);
        s.initVerify (key);
        s.update (data.getBytes());
        return s.verify (signature);
    }
}

```

}
